

REV	DATA	ZMIANY
0.1	19.05.2025	<i>Michał Ferens (sina@student.agh.edu.pl)</i>
1.0	16.06.2025	<i>Michał Ferens (sina@student.agh.edu.pl)</i>

KROKOMIERZ Z KOMUNIKACJĄ BEZPRZEWODOWĄ

Autor: Michał Ferens

Prowadzący: Prof. dr hab. inż. Bogusław Cyganek

Akademia Górniczo-Hutnicza

Spis treści

1.	WSTĘP.....	4
2.	FUNKCJONALNOŚĆ (<i>FUNCTIONALITY</i>).....	6
3.	ANALIZA PROBLEMU (<i>PROBLEM ANALYSIS</i>)	7
4.	PROJEKT TECHNICZNY (<i>TECHNICAL DESIGN</i>)	9
5.	OPIS REALIZACJI (<i>IMPLEMENTATION REPORT</i>)	12
5.1	POŁĄCZENIA	12
5.2	ARCHITEKTURA OPROGRAMOWANIA	12
5.3	GŁÓWNA PĘTLA PROGRAMU	13
6.	OPIS WYKONANYCH TESTÓW (<i>TESTING REPORT</i>)	15
6.1	TEST KOMUNIKACJI I2C.....	15
6.2	TEST ODCZYTU DANYCH Z AKCELEROMETRU	16
6.3	TEST OBLICZANIA MODUŁU PRZYSPIESZENIA.....	17
6.4	TEST DETEKЦИИ KROKÓW.....	18
6.5	TEST ADVERTISING BLE	19
6.6	TEST GATT	20
6.7	TEST BLE NOTIFY	21
7.	PODRĘCZNIK UŻYTKOWNIKA (<i>USER'S MANUAL</i>).....	21
7.1	URUCHOMIENIE SYSTEM	21
7.2	POŁĄCZENIE Z URZĄDZENIEM	21
8.	METODOLOGIA ROZWOJU I UTRZYMANIA SYSTEMU (<i>SYSTEM MAINTENANCE AND DEPLOYMENT</i>).....	22
9.	KIERUNKI DALSZEGO ROZWOJU PROJEKTU	23
	BIBLIOGRAFIA.....	24

Lista oznaczeń

BLE	Bluetooth Low Energy
DSP	Digital Signal Processing
RTOS	Real-Time Operating System
MEMS	Micro-Electro-Mechanical System

1. Wstęp

Dokument dotyczy opracowania i implementacji mikroprocesorowego krokomierza z bezprzewodową transmisją danych. Celem projektu jest stworzenie urządzenia, które będzie zliczać kroki wykonane przez użytkownika oraz przysyłać zagregowane dane do zewnętrznej aplikacji mobilnej z wykorzystaniem technologii BLE.

1. Wymagania systemowe (*requirements*)

Podstawowe założenia projektu:

Jednostka obliczeniowa: 32-bitowy mikrokontroler z rodziny ESP32, odpowiedzialny za zarządzanie peryferiami oraz realizację stosu komunikacyjnego.

Akwizycja danych: układ MEMS (np. MPU-6050) integrujący akcelerometr i żyroskop, komunikujący się z jednostką centralną za pośrednictwem magistrali I2C.

Algorytmika: Zaprojektowanie i implementacja autorskiego algorytmu detekcji kroków z wykorzystaniem metod cyfrowego przetwarzania sygnałów (DSP), obejmującego wstępną filtrację danych oraz detekcję wartości progowych.

Komunikacja: protokół BLE asynchronicznie przesyłający wyniki pomiarów do urządzeń zewnętrznych.

Architektura oprogramowania: System czasu rzeczywistego FreeRTOS oraz środowisko ESP-IDF, Zapewni to współbieżność zadań oraz efektywne zarządzanie zasobami i energią mikrokontrolera.

Zarządzanie kodem: GIT

Potrzebne elementy:

- Płytki deweloperskie ESP32 (np. ESP-WROOM-32 USB-C CH340 WiFi BT)
- Moduł czujnika (np. MPU-6050)
- Płytki stykowe i przewody

2. Funkcjonalność (*functionality*)

- Urządzenie ma na bieżąco monitorować odczyty z czujnika. Obliczany będzie wektor wypadkowy w przestrzeni trójwymiarowej. Dzięki temu kroki będą zliczane niezależnie, w którą stronę idziemy lub jak ułożony jest sensor.
- Możliwe ma być bezprzewodowe połączenie się z krokomierzem za pomocą telefonu. Dzięki temu bezprzewodowo będzie można odczytać liczbę kroków naliczoną przez krokomierz. Ma też być możliwość zresetowania liczby kroków z telefonu
- System ma automatycznie zarządzać zużyciem energii tak, żeby był jak najbardziej energooszczędny.
- Oprogramowanie ma bazować na systemie operacyjnym FreeRTOS. System ma działać tak, żeby nawet w przypadku wydłużonego czasu potrzebnego na połączenie się z telefonem odczytywał zawsze dane z sensora. Dzięki temu nie będzie pominięty żaden krok. Zrealizowane to będzie dzięki wątkom o różnych priorytetach.
- Kod napisany w C++ dzięki swojej obiektowości ma zapewnić rozszerzalność i łatwość w modyfikacji oraz niezależność poszczególnych części (np. algorytmu detekcji od obsługi magistrali).

3. Analiza problemu (*problem analysis*)

Akwizycja i analiza ruchu ludzkiego ciała podczas chodu, wymaga formalnego zdefiniowania reprezentacji matematycznej mierzonych wielkości fizycznych. Sygnał wejściowy rejestrowany przez akcelerometr w układzie MP-6050 jest dyskretnym sygnałem wektorowym, zdefiniowanym w dziedzinie czasu dyskretnego n :

$$s[n] = \begin{bmatrix} x & [n] \\ y & [n] \\ z & [n] \end{bmatrix}$$

gdzie wartości $x[n], y[n], z[n] \in \mathbb{R}$ oznaczają chwilowe wartości przyspieszenia liniowego zarejestrowane odpowiednio w osiach podłużnej, poprzecznej i pionowej sensora w chwili próbkowania n . Każda z próbek składowych jest funkcją zależną od czasu obciążoną szumem wysokoczęstotliwościowym, który wynika z drgań mechanicznych oraz szumów własnych czujnika, dodatkowo w grę wchodzi składowa stała, która reprezentuje przyspieszenie ziemskie g .

Głównym problemem w implementacji krokomierzy jest zmienna i nieprzewidywalna orientacja przestrzenna czujnika względem wektora grawitacji. Jest to spowodowane tym, że zazwyczaj taki krokomierz nosimy na ręku w postaci zegarka, lub w kieszeni czy w plecaku i po prostu ciągle będzie się on obracał i generalnie zmieniał trochę swoje położenie. Gdy to się dzieje, podczas chodzenia składowe wektora grawitacji rozkładają się dynamicznie i nierównomiernie na wszystkich osiach (x, y, z). Próba analizy tylko jednej z nich prowadziłaby do krytycznych błędów pomiarowych i gubienia kroków w sytuacji, gdy urządzenie ulegnie rotacji o 90 stopni.

Aby uniezależnić algorytm detekcji od orientacji układu, w pierwszej fazie DSP wyznaczany jest moduł wektora wypadkowego sygnału trójwymiarowego:

$$a_{mag}[n] = \|s[n]\| = \sqrt{x[n]^2 + y[n]^2 + z[n]^2}$$

Sygnał $a_{mag}[n]$ jest sygnałem jednowymiarowym, reprezentującym całkowite przyspieszenie działające na układ, pozbawionym składowych rotacyjnych. W stanie spoczynku urządzenia, wartość modułu wypadkowego powinna oscylować wokół wartości przyspieszenia ziemskiego:

$$a_{mag}[n] \approx g \approx 9.81 \frac{m}{s^2}$$

Podczas chodzenia, moment, w którym odpychamy się nogą od ziemi (przeciwstawiamy się grawitacji), generowany jest impuls mechaniczny, który jest rejestrowany przez sensor jako wyraźne, periodyczne maksima o amplitudzie wyraźnie przekraczającej poziom składowej stałej g . Powinno to być też w tym samym kierunku, ale przeciwnym zwrocie niż wektor wynikający z przyspieszenia ziemskiego. Zjawisko to pozwala na zastosowanie algorytmu detekcji przejść przez zero.

Proces przetwarzania sygnału w ujęciu systemowym wbudowanym (FreeRTOS) realizuje poniższa sekwencja operacji matematyczno-logicznych:

1. Pobranie wektora $s[n]$ z rejestrów MPU-6050 przez magistralę I2C z częstotliwością próbkowania $f_s = 50$ Hz.
2. Wyznaczenie wartości $a_{mag}[n]$ w celu eliminacji wpływu obrotu obudowy.
3. Zastosowanie cyfrowego filtra dolnoprzepustowego.
4. Porównanie przefiltrowanego sygnału z dynamicznie adaptowanym lub stałym progiem czułości $a_{threshold}$.
5. Wprowadzenie histerezy czasowej, na przykład w postaci blokady ponownej detekcji przez minimum 250 ms po wykryciu kroku. Pozwoli to na uniknięcie wielokrotnego zliczania tego samego kroku.

4. Projekt techniczny (*technical design*)

Poniższe diagramy przedstawiają koncepcję architektury systemu przygotowaną na etapie projektowania. W trakcie implementacji zakres projektu został zawężony do działającego prototypu realizującego główne funkcje: pomiar przyspieszenia, detekcję kroków oraz transmisję liczby kroków przez BLE. Funkcje takie jak resetowanie licznika z poziomu telefonu, automatyczne usypianie oraz rozdzielenie programu na kilka niezależnych zadań FreeRTOS pozostawiono jako możliwe rozszerzenia systemu.

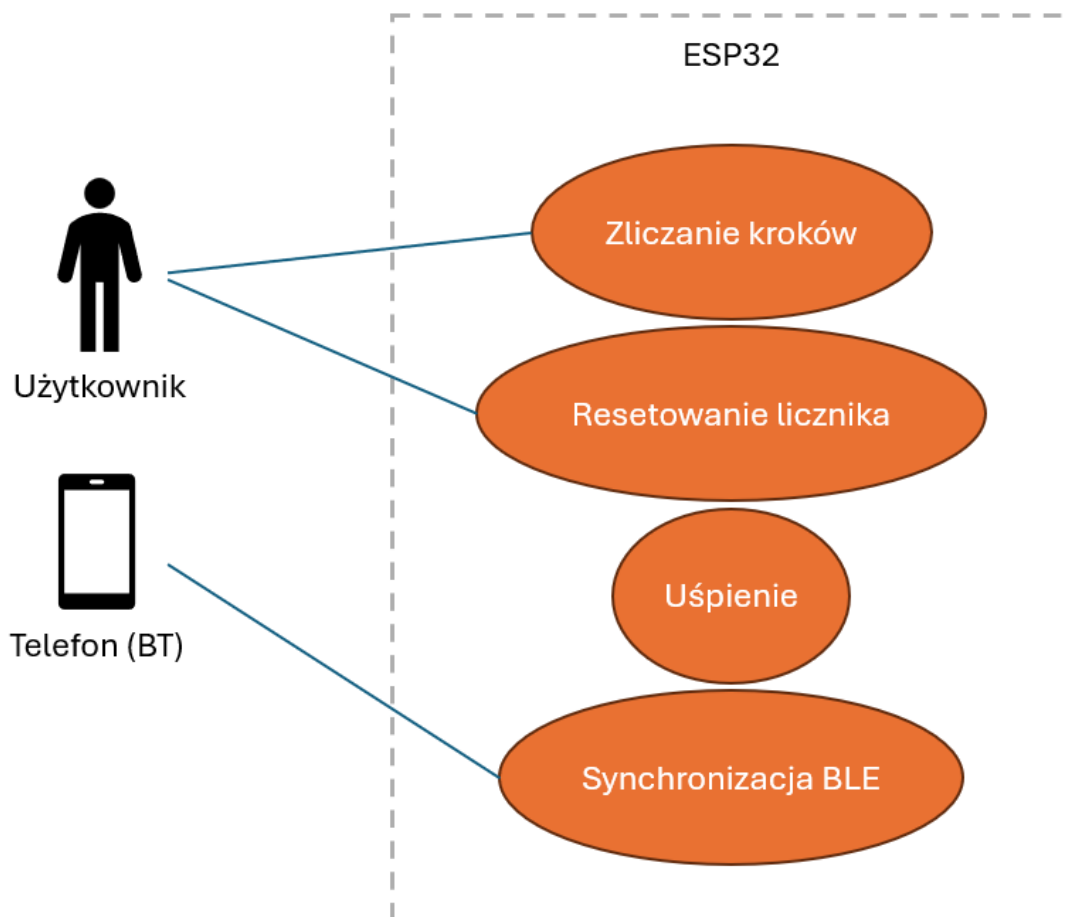


Figure 1 - use case diagram (UML)

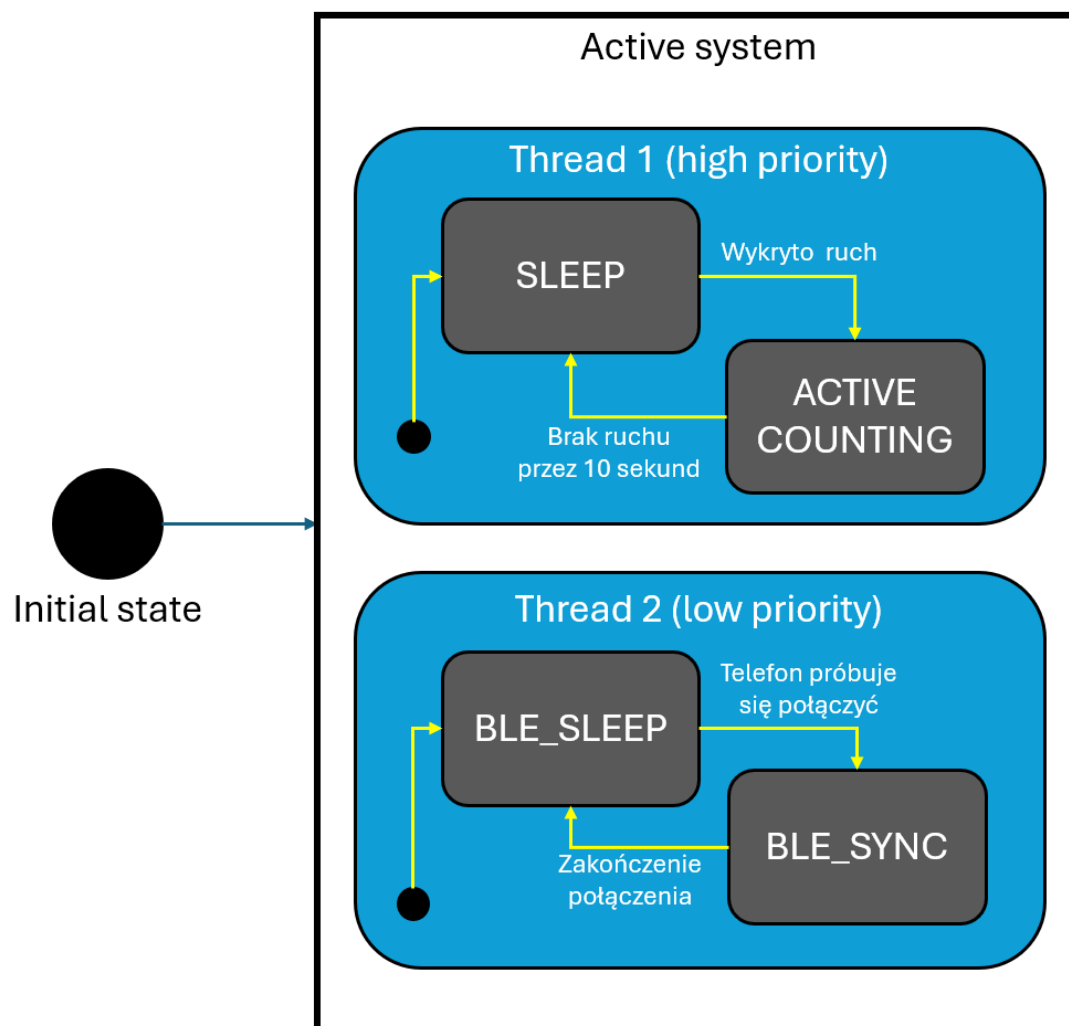


Figure 2 - state machine diagram (UML)

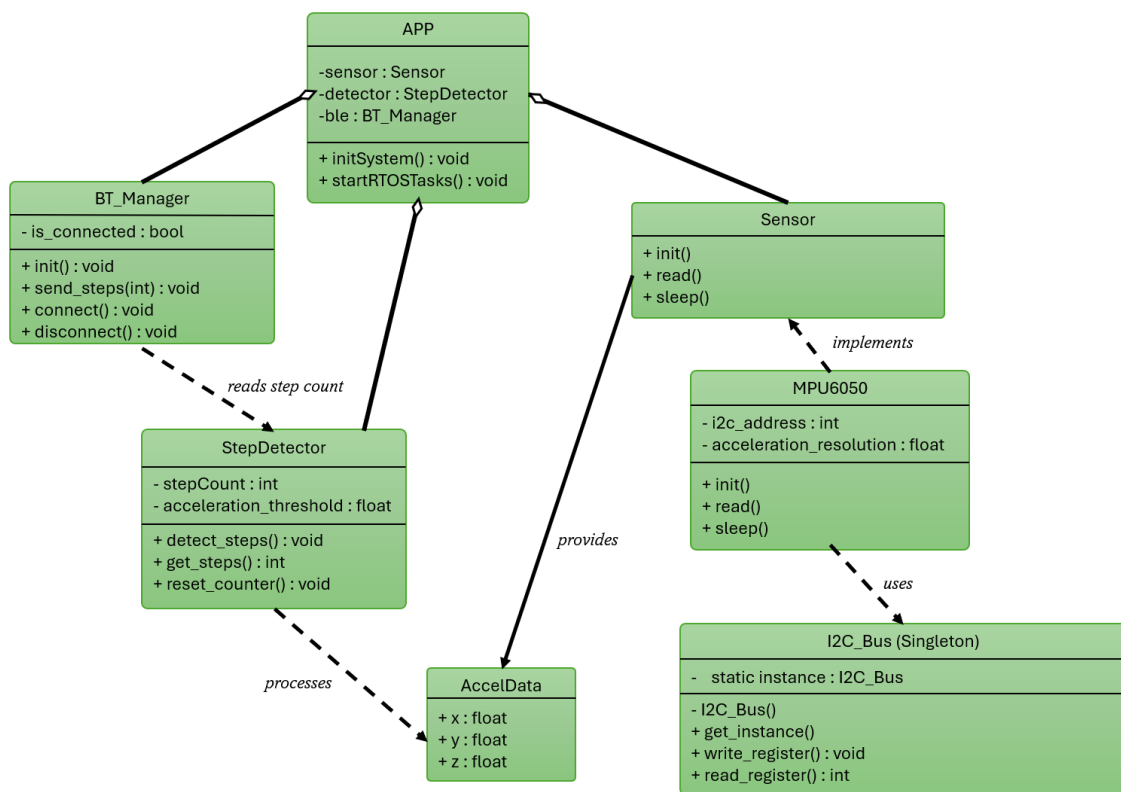


Figure 3 - class diagram (UML)

5. Opis realizacji (*implementation report*)

Projekt został zrealizowany z wykorzystaniem mikrokontrolera ESP-WROOM-32 USB-C CH340 WiFi BT oraz modułu MPU6050 zawierającego trójosiowy akcelerometr. Oprogramowanie zostało napisane w języku C++ w środowisku ESP-IDF 6.0.1 z wykorzystaniem systemu operacyjnego czasu rzeczywistego FreeRTOS. Kod źródłowy był rozwijany w środowisku Visual Studio Code oraz przechowywany w repozytorium Git.

5.1 Połączenia

Komunikacja pomiędzy ESP32 i MPU6050 została zrealizowana za pomocą magistrali I2C. Czujnik został podłączony w następujący sposób:

MPU6050	ESP32
SDA	GPIO21
SCL	GPIO22
VCC	3.3 V
GND	GND

5.2 Architektura oprogramowania

Oprogramowanie zostało podzielone na kilka niezależnych modułów odpowiedzialnych za poszczególne funkcjonalności systemu. W stosunku do wstępnych diagramów UML implementacja została uproszczona. Zrezygnowano z osobnej warstwy abstrakcji Sensor oraz z funkcji usypiania, ponieważ głównym celem projektu było uzyskanie działającego prototypu krokomierza z komunikacją BLE. Moduły zostały jednak zachowane w formie klas C++, co umożliwia późniejsze rozszerzanie systemu.

Klasa I2C_Bus odpowiada za konfigurację oraz obsługę magistrali I2C. Udostępnia funkcje umożliwiające zapis i odczyt pojedynczych rejestrów oraz odczyt wielu kolejnych bajtów z urządzenia peryferyjnego.

W celu zapewnienia dostępu do jednej wspólnej instancji magistrali zastosowano wzorzec projektowy Singleton.

Klasa MPU6050 odpowiada za komunikację z czujnikiem przyspieszenia.

Do najważniejszych funkcji należą:

- inicjalizacja sensora,
- odczyt surowych danych akcelerometru,
- konwersja wyników na jednostki g,
- wyznaczenie modułu wektora przyspieszenia,
- identyfikacja układu za pomocą rejestru WHO_AM_I.

Klasa StepDetector realizuje algorytm wykrywania kroków.

Algorytm bazuje na obserwacji modułu przyspieszenia wyznaczanego ze wzoru:

Jeżeli wartość przyspieszenia przekroczy ustalone progi detekcji, a od poprzedniego wykrytego kroku upłynie odpowiedni czas blokady, licznik kroków zostaje zwiększony.

Zastosowana histereza czasowa zapobiega wielokrotnemu zliczaniu tego samego kroku.

Klasa BLE_Manager odpowiada za konfigurację i obsługę komunikacji Bluetooth Low Energy.

W projekcie utworzono własną usługę GATT:

- Service UUID: FFF0
- Characteristic UUID: FFF1

Charakterystyka FFF1 przechowuje aktualną liczbę kroków.

Dane mogą być przesyłane na dwa sposoby:

- odczyt (Read),
- automatyczne powiadomienia (Notify).

Dzięki temu liczba kroków jest automatycznie aktualizowana w aplikacji mobilnej bez konieczności ręcznego odświeżania.

5.3 Główna pętla programu

Po uruchomieniu systemu wykonywane są następujące operacje:

1. Inicjalizacja magistrali I2C.
2. Inicjalizacja czujnika MPU6050.
3. Inicjalizacja komunikacji BLE.
4. Uruchomienie głównej pętli programu.

W pętli wykonywane są kolejno:

1. Odczyt danych z akcelerometru.
2. Obliczenie modułu przyspieszenia.
3. Detekcja kroku.
4. Aktualizacja licznika kroków.
5. Wysłanie nowej wartości do telefonu za pomocą BLE.

Próbkowanie sygnału realizowane jest okresowo z wykorzystaniem funkcji FreeRTOS *vTaskDelay()*.

6. Opis wykonanych testów (*testing report*)

6.1 Test komunikacji I2C

Cel testu: Weryfikacja poprawności komunikacji pomiędzy ESP32 i czujnikiem MPU6050

Przebieg testu: Po uruchomieniu wykonano odczyt rejestru identyfikacyjnego WHO_AM_I znajdującego się pod adresem 0x75

Oczekiwany wynik: Zwrócona wartość rejestru 0x68

Rezultat:

```
// Funkcje testowe używane podczas uruchamiania projektu.  
| sensor.readWhoAmI();  
// sensor.readAcceleration();  
// sensor.readAccelerationG();  
// sensor.getAccelerationMagnitude();
```

▼ TERMINAL

```
I (431) BTDM_INIT: Using main XTAL as clock source  
I (431) BTDM_INIT: Bluetooth MAC: d4:e9:f4:78:3e:22  
I (441) phy_init: phy_version 4863,a3a4459,Oct 28 2025,14:30:06  
[BLE_Manager] GATT services registered.  
[BLE_Manager] BLE host task started.  
[BLE_Manager] NimBLE uruchomione poprawnie.  
[I2C_Bus] Odczyt OK: device=0x68, reg=0x75, data=0x68  
[MPU6050] WHO_AM_I = 0x68  
[BLE_Manager] BLE synchronized.  
I (821) NimBLE: GAP procedure initiated: advertise;  
I (821) NimBLE: disc_mode=2  
I (821) NimBLE: adv_channel_map=0 own_addr_type=0 adv_filter_policy=0 adv_itvl_min=0 a  
dv_itvl_max=0  
I (821) NimBLE:
```

6.2 Test odczytu danych z akcelerometru

Cel testu: Sprawdzenie poprawności odczytu danych przyspieszenia z trzech osi sensora.

Przebieg testu: Uruchomiono funkcję odczytu danych akcelerometru oraz obserwowano zmiany wartości podczas poruszania modulem.

Oczekiwany wynik: Podczas gdy sensor leży na biurku to powinno być widoczne głównie przyspieszenie ziemskie w jednej osi. Podczas poruszania zmiany wartości.

Rezultat:

6.3 Test obliczania modułu przyspieszenia

Cel testu: Sprawdzenie poprawności wyznaczania modułu wektora przyspieszenia

Przebieg testu: Urządzenie pozostawiono nieruchomo na płaskiej powierzchni.

Oczekiwany wynik: Wartość modułu przyspieszenia powinna być zbliżona do 1 g.

Rezultat:

```
40
41  while (true)
42  {
43      sensor.getAccelerationMagnitude();
44
45      // Odczyt aktualnego przyspieszenia i wyznaczenie
46      // wartości wypadkowej (magnitude).
47      // float magnitude = sensor.getAccelerationMagnitude();
48
49      // // Sprawdzenie, czy algorytm wykrył krok.
50      // if (detector.detectStep(magnitude))
51      // {
52      //     // Pobranie aktualnej liczby kroków.
53      //     uint32_t steps = detector.getStepCount();
54
55      //     // Aktualizacja wartości udostępnianej przez BLE.
56      //     ble.updateStepCount(steps);
57
58      //     // Wysłanie nowej wartości do telefonu (BLE Notify
59      //     ble.notifyStepCount();
60
61      //     printf("KROK! Liczba krokow: %lu\n", steps);
62      // }
63
64      // Odczyt czujnika wykonywany co 50 ms.
65      vTaskDelay(500 / portTICK_PERIOD_MS);
66  }
67 }
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS ESP-IDF

TERMINAL

```
1.125
1.127
1.115
1.123
1.125
1.122
1.124
1.119
1.122
1.124
```

6.4 Test detekcji kroków

Cel testu: Weryfikacja poprawności działania liczenia kroków

Przebieg testu: Użytkownik wykonuje kilka/kilkanaście kroków trzymając moduł ESP z sensorem i liczy je sobie w myślach.

Oczekiwany wynik: Liczba kroków, które zrobił użytkownik powinna pokrywać się z tą na liczniku

Rezultat: Liczba wykonanych kroków – 10

```
0.895
1.245
KROK! Liczba krokow: 2
1.384
KROK! Liczba krokow: 3
1.101
1.003
1.061
0.845
1.149
1.242
KROK! Liczba krokow: 4
1.221
KROK! Liczba krokow: 5
1.099
1.108
1.113
1.246
KROK! Liczba krokow: 6
1.025
1.047
1.413
KROK! Liczba krokow: 7
0.998
1.038
0.813
1.117
1.329
KROK! Liczba krokow: 8
1.118
1.025
```

Liczba policzonych kroków – 8

Do poprawy progi/delay

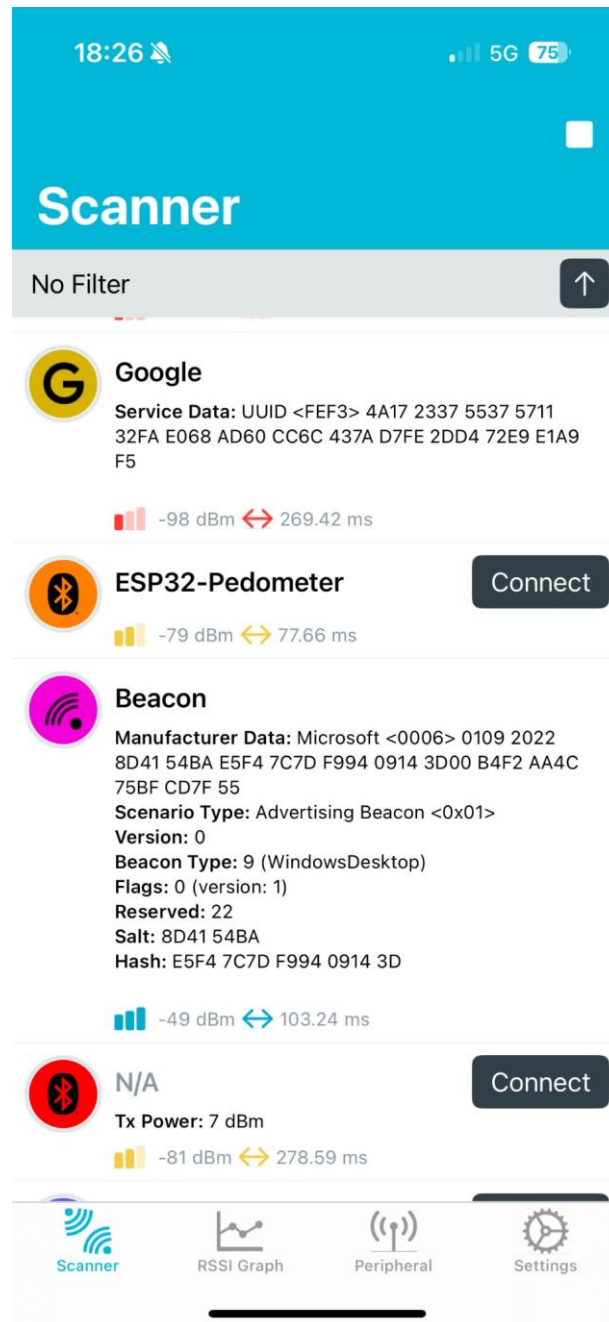
6.5 Test advertising BLE

Cel testu: Sprawdzenie widoczności urządzenia

Przebieg testu: Uruchomiono nRF Connect na telefonie i wykonano skanowanie urządzeń.

Oczekiwany wynik: Urządzenie powinno być widoczne pod nazwą “ESP32-Pedometer”

Rezultat:



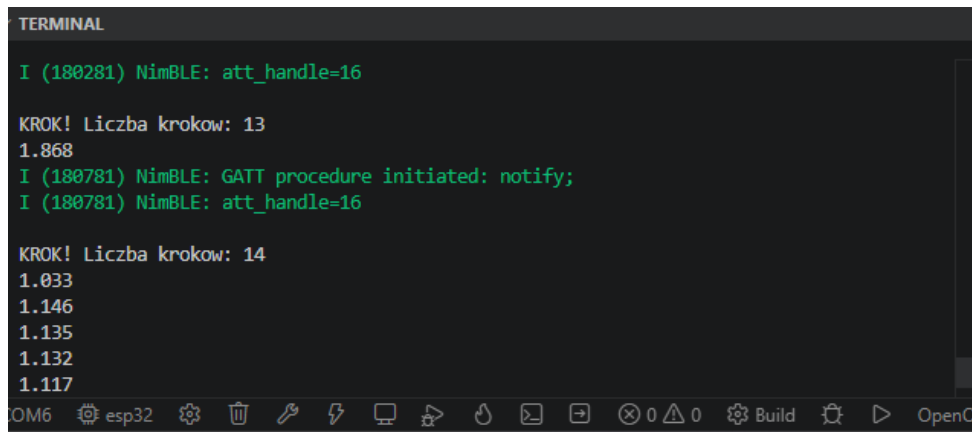
6.6 Test GATT

Cel testu: Sprawdzenie poprawności odczytu liczby kroków na telefonie

Przebieg testu: Po połączeniu wykonano odczyt charakterystyki FFF1.

Oczekiwany wynik: Liczba kroków na telefonie powinna pokrywać się z tą na terminal

Rezultat:

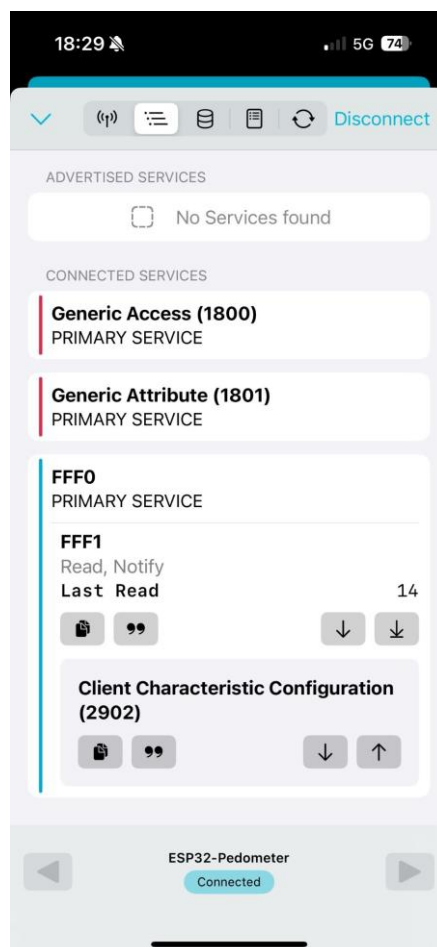


```
TERMINAL

I (180281) NimBLE: att_handle=16

KROK! Liczba krokow: 13
1.868
I (180781) NimBLE: GATT procedure initiated: notify;
I (180781) NimBLE: att_handle=16

KROK! Liczba krokow: 14
1.033
1.146
1.135
1.132
1.117
```



6.7 Test BLE Notify

Cel testu: Weryfikacja automatycznej aktualizacji liczby kroków bez wykonywania operacji Read

Przebieg testu: Po połączeniu z urządzeniem wykonywano kolejne kroki obserwując wartość charakterystyki FFF1 w aplikacji nRF Connect.

Oczekiwany wynik: Wartość charakterystyki powinna aktualizować się automatycznie po każdym wykrytym kroku.

Rezultat: Dostępny na GitHub w folderze tests

7. Podręcznik użytkownika (*user's manual*)

7.1 Uruchomienie system

W celu uruchomienia system należy podłączyć ESP do zasilania (np. przez USB do powerbanka). Po uruchomieniu mikrokontroler automatycznie inicjalizuje czujnik MPU6050 oraz uruchamia komunikację Bluetooth Low Energy.

7.2 Połączenie z urządzeniem

W celu nawiązania połączenia wykorzystano aplikację nRF connect (można ją pobrać na Sklep Play albo App Store).

- I. Uruchomić aplikację nRF Connect
- II. Rozpocząć skanowanie urządzeń
- III. Odnaleźć urządzenie "ESP32-Pedometer"
- IV. Wybrać Connect
- V. Wybrać opcję notify 
- VI. Wybrać odpowiedni format UTF-8 (zmiany format można dokonać klikając )

Po poprawnym połączeniu aplikacja wyświetla charakterystykę FFF1 zawierającą aktualną liczbę kroków.

8. Metodologia rozwoju i utrzymania systemu (*system maintenance and deployment*)

Projekt został zaimplementowany w języku C++ z wykorzystaniem środowiska ESP-IDF. Funkcjonalność systemu została podzielona na niezależne moduły reprezentowane przez klasy I2C_Bus, MPU6050, StepDetector oraz BLE_Manager. Takie podejście zwiększa czytelność kodu oraz ułatwia jego dalszy rozwój i utrzymanie. Podczas implementacji wykorzystano wzorzec projektowy Singleton w klasie I2C_Bus. Dzięki temu w całym systemie istnieje tylko jedna instancja magistrali I2C odpowiedzialna za komunikację z urządzeniami peryferyjnymi.

Kod źródłowy ma komentarze opisujące działanie klas, metod oraz najważniejszych fragmentów algorytmów. Zastosowano również podejście self-documenting code poprzez używanie jednoznacznych nazw klas, metod i zmiennych.

Projekt był rozwijany z wykorzystaniem systemu kontroli wersji Git. Repozytorium umożliwiała śledzenie zmian, tworzenie kolejnych wersji oprogramowania oraz bezpieczne rozwijanie nowych funkcjonalności.

W trakcie realizacji przeprowadzano regularne testy poszczególnych modułów systemu. Testowano poprawność komunikacji I2C, odczytu danych z MPU6050, działania algorytmu detekcji kroków oraz komunikacji Bluetooth Low Energy. Wyniki testów zostały przedstawione w rozdziale 6.

9. Kierunki dalszego rozwoju projektu

Obecna wersja projektu stanowi działający prototyp realizujący podstawową funkcjonalność krokomierza BLE. W kolejnych etapach rozwoju planowane jest rozszerzenie systemu zgodnie z przedstawionymi diagramami UML.

Planowane funkcjonalności obejmują:

- implementację trybu uśpienia ograniczającego pobór energii przy braku ruchu,
- możliwość resetowania licznika kroków z poziomu aplikacji mobilnej,
- rozbudowę komunikacji BLE o dodatkowe charakterystyki konfiguracyjne,
- rozbudowę maszyny stanów systemu zgodnie z diagramem UML. Obecna wersja działa głównie w trybie aktywnego pomiaru, natomiast wersja docelowa powinna rozróżniać stany takie jak SLEEP, ACTIVE_COUNTING, BLE_SLEEP oraz BLE_SYNC,
- wydzielenie niezależnych zadań FreeRTOS odpowiedzialnych za pomiary i komunikację BLE. W obecnej wersji główna pętla programu realizuje odczyt sensora i detekcję kroków, natomiast NimBLE pracuje w osobnym zadaniu systemowym. W wersji docelowej można utworzyć osobne zadanie o wyższym priorytecie dla akwizycji danych oraz osobne zadanie o niższym priorytecie dla komunikacji BLE,
- dopracowanie algorytmu detekcji kroków przez poprawienie progów oraz częstotliwości pomiarów oraz minimalnego delayu pomiędzy kolejnymi krokami. Następnie dodanie cyfrowej filtracji sygnału,
- zapis historii kroków w pamięci nieulotnej ESP32,
- prezentację danych w dedykowanej aplikacji mobilnej zamiast aplikacji diagnostycznej nRF Connect,
- utworzenie warstwy abstrakcji sensora zgodnie z diagramem klas. W obecnej wersji wykorzystywana jest bezpośrednio klasa MPU6050. Wersja docelowa może zawierać ogólny interfejs Sensor, który będzie implementowany przez MPU6050. Ułatwi to późniejszą wymianę czujnika na inny model bez konieczności modyfikowania algorytmu detekcji kroków,
- dopracowanie zasilania bateryjnego. Podczas prób z powerbankiem zaobserwowano automatyczne wyłączanie zasilania prawdopodobnie spowodowane zbyt niskim poborem prądu przez układ. W przyszłości należy zastosować źródło zasilania przeznaczone do małych obciążeń lub zaprojektować dedykowany układ zasilania bateryjnego.

Bibliografia

- [1] Cyganek B.: Programowanie w języku C++. Wprowadzenie dla inżynierów. PWN, 2023.
- [2] Mikrobot: Karta informacyjna: Mikrokontroler ESP-WROOM-32 USB-C CH340 WiFi BT (wrzucona do git)
- [3] Mikrobot: Karta informacyjna: MPU 6050 Żyroskop GY-521 Akcelerometr Arduino 3axi (wrzucona do git)
- [4] [TDK InvenSense, MPU-6000 and MPU-6050 Register Map and Descriptions.](#)